Jonathan Dereje

MSSE 601

Fall 2026

Week 5 Owner: Architectural Approach and Object Model

October 11, 2026

Portfolio Decision Record: Choosing the Golden Bike Shop Architecture

Context

Golden Bike Shop needed a system to coordinate retail, inventory, rental reservations, repairs, service bays, and store pickups across four locations. The system would manage approximately 400 rental bikes and needed consistent information so that a bike under repair could not be reserved and service bays could not be double-booked. Week 3 prioritized data consistency, security, performance, availability, and usability, including support for up to 100 concurrent users, three-second availability searches for at least 95% of requests, and cross-store updates within ten seconds for at least 99% of updates. I owned the Week 5 Architectural Approach and Object Model sections and documented how the architecture supported the Week 3 requirements and Week 4 user journeys.

Options

A microservices architecture would allow the inventory, rental, and repair functions to be deployed and scaled independently. However, it would also require more service communication, monitoring, deployment automation, distributed transaction management, and failure handling. A fully event driven architecture would support asynchronous processing and reduce direct dependencies, but eventual consistency could make it more difficult to coordinate reservations, inventory, repairs, and service bay appointments. The modular monolith provided a third option: logical separation between business functions while keeping the core transactions within one application and database. We selected the modular monolith because it provided the strongest balance between consistency, simplicity, and the team's ability to operate the design within the project's constraints.

Decision

I supported selecting a modular n-tier monolithic architecture with asynchronous workers and one shared relational database. Customers, store employees, and store managers use role-based web interfaces connected to a Web/API layer. The application contains Inventory and Retail, Rental and Reservations, Repairs and Service Bays, Availability and Business Rules, and Audit Logging modules. The modules share one authoritative database, while asynchronous workers handle notifications, scheduled reports, audit processing, and reconciliation.

Reasoning

The shared relational database supports the highest-priority Week 3 data-consistency requirement by keeping inventory, reservations, orders, repairs, and service bays in one authoritative location. Centralized transactions help prevent overlapping reservations, rental bikes under repair appearing as available, and double-booked service bays. This directly supports the Week 4 rental, online-order, and repair screens.

The architecture also fits the project's scale and team situation. Four stores, 400 rental bikes, and up to 100 concurrent users did not demonstrate a need for independently deployed services. A modular monolith reduces the deployment and monitoring effort required from a small team, while asynchronous workers prevent noncritical follow-up work from delaying core transactions.

We intentionally prioritized simpler coordination and stronger consistency over independent deployment and scaling because data consistency was the highest-priority Week 3 quality attribute.

Consequences

The architecture simplifies shared transactions and cross-store consistency, but it couples deployments, limits independent scaling, and leaves the shared database as a potential bottleneck and system-wide failure point. Meeting the Week 3 availability target would require additional redundancy, failover, monitoring, and recovery planning.

The team accepted these costs because microservices would add operational complexity without a demonstrated need at the project's expected scale. I would revisit the decision if performance testing failed to meet the three-second target, traffic grew substantially beyond the 100-user assumption, a module required independent scaling, or the business developed a team capable of operating distributed services.

Evidence

Week owned: Week 5

Deliverable: Week 5 Project Component: Application Architecture

Ownership: Architectural Approach and Object Model

Location: "Architectural Approach" and "Object Model" sections

AI contribution: AI stress-tested the selected architecture and identified the shared database and single deployment as possible performance bottlenecks and broad failure points. I checked the objection against the Week 3 quality requirements and kept the architecture while clarifying module boundaries, synchronous processing for reservations and inventory, and asynchronous processing for notifications, reports, audit processing, and reconciliation.